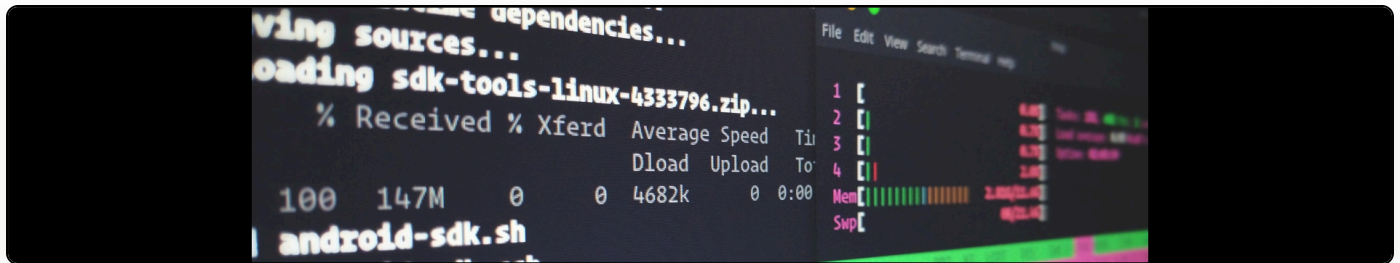


Handling the absence of a (newline) break tag in JATS

Cite as: Martin Paul Eve, "Handling the absence of a (newline) break tag in JATS", *eve.gd: Martin Paul Eve*, January 13, 2014, <https://doi.org/10.59348/485d3-zd498>.



As I noted in a previous post, a lot of my work this term involves technical implementation of an open source JATS (previously NLM) typesetter for scholarly articles. What this means is that I am writing a system that takes imperfectly formatted Microsoft Word documents and transforms them into an XML format that we can use to produce HTML, PDFs, EPUBs you name it. I'm intending to write about my experience of developing this system as a way of ensuring my thoughts are clear but also as a way in for anybody else who might ever want to understand the meTypeset codebase.

One of the problems that I dealt with today is the fact that the JATS standard does not allow for a line-break to occur mid-way through a paragraph. This will occur in a Word document when the user types a paragraph and then presses control + enter. So it isn't a new paragraph, it instead is just a new line within a paragraph. In XHTML this would be presented like this:

```
<p>A line of text<br/>another line of text</p>
```

JATS has no equivalent to the br tag mid-way through that line.

The way that I've decided to handle this in meTypeset is via an option flag in the configuration file:

```
<mt:linebreaks-as-comments>False</mt:linebreaks-as-comments>
```

When this is set to True, meTypeset inserts a commented-out variable that indicates to subsequent transforms that they should insert a line-break here to ensure fidelity to the original document. The comment tag is `<!--meTypeset:br-->`.

When this is set to False, we treat the `<!--meTypeset:br-->` tags as though they should be changed into a `</p><p>` sequence (close-paragraph, open-paragraph).

This sounds simple, but there are some dangerous problems. Consider what happens in this scenario:

```
<p>A line of text<!--meTypeset:br-->another line of tex</p>
```

If you just transform that `<!--meTypeset:br-->` into a `</p><p>`, you get a hideously malformed mess that looks like this:

```
<p>A line of <italic>text</p><p>another</italic> line of text</p>
```

So, what we have to do is to build a stack of all the elements applied to the tail after the comment() element in that XPath. This is implemented in [our NLMManipulator class](#).

The relevant method that styles this is `handle_nested_elements`:

```

@staticmethod
def handle_nested_elements(iter_node, move_node, node, node_parent, outer_node,
tag_name, tail_stack,
                        tail_stack_objects):
    while iter_node.tag != tag_name:
        tail_stack.append(iter_node.tag)
        tail_stack_objects.append(iter_node)
        iter_node = iter_node.getparent()

    # get the tail (of the comment) and style it
    append_location = None
    tail_text = node.tail
    iterator = 0
    tail_stack.reverse()
    tail_stack_objects.reverse()
    # rebuild the styled tree on a set of subelements
    for node_to_add in tail_stack:
        sub_element = etree.Element(node_to_add)
        if iterator == len(tail_stack) - 1:
            sub_element.text = node.tail

        if iterator == 0:
            outer_node = sub_element

        iterator += 1
        if append_location is None:
            tail_stack_objects[0].addnext(sub_element)
            append_location = sub_element
        else:
            append_location.append(sub_element)
            append_location = sub_element

    # remove the old node (this is in the element above)
    node.getparent().remove(node)
    # set the search node to the outermost node so that we can find siblings
    node_parent = iter_node
    node = outer_node
    move_node = True
    return move_node, node, node_parent

```

This builds a list of elements working outwards before re-adding the content of the tail of the last processed node using lxml.

Combined with the `search_and_copy` and `process_node_for_tags` method, this has worked in the limited cases that I've thrown at it. I'm sure there are bugs to be uncovered, but this is the start of our handling of this scenario.